

Digital Policy Office

INFORMATION SECURITY

Practice Guide for Website and Web Application Security

Version 1.2

July 2024

© The Government of the Hong Kong Special Administrative Region
of the People's Republic of China

The contents of this document remain the property of and may not be reproduced in whole or in part without the express permission of the Government of the Hong Kong Special Administrative Region of the People's Republic of China.

COPYRIGHT NOTICE

© 2024 by the Government of the Hong Kong Special Administrative Region of the People's Republic of China

Unless otherwise indicated, the copyright in the works contained in this publication is owned by the Government of the Hong Kong Special Administrative Region of the People's Republic of China. You may generally copy and distribute these materials in any format or medium provided the following conditions are met –

- (a) the particular item has not been specifically indicated to be excluded and is therefore not to be copied or distributed;
- (b) the copying is not done for the purpose of creating copies for sale;
- (c) the materials must be reproduced accurately and must not be used in a misleading context; and
- (d) the copies shall be accompanied by the words “copied/distributed with the permission of the Government of the Hong Kong Special Administrative Region of the People's Republic of China. All rights reserved.”

If you wish to make copies for purposes other than that permitted above, you should seek permission by contacting the Digital Policy Office.

Amendment History				
Change Number	Revision Description	Pages Affected	Revision Number	Date
1	Updates were made on personal data protection, secure testing with source code scan and recording and monitoring access log	3.3, 3.4.2 Annex B Annex D	1.1	June 2021
2	Change “Office of the Government Chief Information Officer” (or “OGCIO”) to “Digital Policy Office” (or “DPO”)		1.2	July 2024

Table of Contents

1.	Introduction.....	1
1.1	Purpose.....	1
1.2	Normative Reference	1
1.3	Terms and convention.....	2
1.4	Contact	3
2.	Website and Web Application Security	4
2.1	Website and Web Application Threats	4
3.	Website and Web Application Life Cycle	11
3.1	Project Initiation.....	11
3.2	Feasibility Study	12
3.3	Systems Analysis & Design.....	12
3.4	Implementation	15
3.5	Post-Implementation	23
3.6	Decommission.....	26
4.	Additional Security Consideration for Web Content Management System and Web 2.0..	28
4.1	Web Content Management System.....	28
4.2	Web 2.0	30
	Annex A: Website and Web Application Security Architecture	1
	Annex B: Web Server Security.....	1
	Annex C: Secure Website with HTTPS Protocol	1
	Annex D: Web Application Secure Coding Practices	1
	Annex E: OWASP Secure Coding Practices Quick Reference Guide	1

1. Introduction

Owing to the common adoption of internet facing websites and website applications by Bureaux and Departments (B/Ds), this practice guide is developed to provide security measures during the software development life cycle to mitigate the common threats to website and web applications.

1.1 Purpose

The purpose of this document is to provide general guidelines and some best practices on website and web application security. It should be used in conjunction with the security requirements and documents such as Baseline IT Security Policy, IT Security Guidelines and relevant procedures, where applicable.

In addition, this practice guide contains technical knowledge in website and web application, the readers are expected to have a basic understanding of technical know-how on infrastructure and/or programming skills. If there is any question on those technical skills, it is suggested to consult with B/Ds' IT team or subject matter experts.

This practice guide is intended for staff who are involved in the life cycle of websites and web applications excluding web users.

1.2 Normative Reference

The following referenced documents are indispensable for the application of this document.

- Security Regulations, Government of the Hong Kong Special Administrative Region
- Practice Guide for Security Risk Assessment & Audit, Government of Hong Kong Special Administrative Region.
- Practice Guide for Information Security Incident Handling, Government of Hong Kong Special Administrative Region.
- Information technology – Security techniques – Information security management systems – Requirements, ISO/IEC 27001:2013
- Information technology – Security techniques – Code of practice for information security controls, ISO/IEC 27002:2013
- Information technology – Security techniques – Governance of information security, ISO/IEC 27014:2013
- Information technology – Security techniques – Application security, ISO/IEC 27034:2015

- Information technology – Security techniques – Storage security, ISO/IEC 27040:2015
- OWASP Top 10 – 2013, The Ten Most Critical Web Application Security Risks, the Open Web Application Security Project
<http://owasptop10.googlecode.com/files/OWASP%20Top%2010%20-%202013.pdf>
- OWASP Secure Coding Practices – Quick Reference Guide, the Open Web Application Security Project
https://www.owasp.org/images/0/08/OWASP_SCP_Quick_Reference_Guide_v2.pdf
- Security Considerations in the System Development Life Cycle, SP 800-64, National Institute of Standards and Technology (NIST)
<http://csrc.nist.gov/publications/nistpubs/800-64-Rev2/SP800-64-Revision2.pdf>
- Implementing Vulnerability Scanning in a Large Organisation, the SANS Institute ;
<https://www.sans.org/reading-room/whitepapers/casestudies/implementing-vulnerability-scanning-large-organisation-1103>
- Technical Guide to Information Security Testing and Assessment, SP 800-115, NIST;
<http://csrc.nist.gov/publications/nistpubs/800-115/SP800-115.pdf>
- Assuring Software Security Through Testing, (ISC)2; and
- TM-E-1 Risk Management of E-banking, Supervisory Policy Manual, the Hong Kong Monetary Authority.
<http://www.hkma.gov.hk/media/eng/doc/key-functions/banking-stability/supervisory-policy-manual/TM-E-1.pdf>

1.3 Terms and convention

For the purposes of this document, the terms and convention given in S17, G3, and the following apply.

Abbreviation and Terms	
NA	NA

1.4 Contact

This document is produced and maintained by the Digital Policy Office (DPO). For comments or suggestions, please send to:

Email: it_security@digitalpolicy.gov.hk

Lotus Notes mail: IT Security Team/DPO/HKSARG@DPO

CMMP email: IT Security Team/DPO

2. Website and Web Application Security

Website¹ or web application² is commonly used to provide services to the public and to the Government staff nowadays. Web 2.0 applications such as Wiki also create a platform for effective knowledge sharing and contribution. Although website or web application provides convenience and efficiency, it is faced with many security threats because the client access can be from anywhere over the Internet.

The threats originate from the untrustworthy client, session-less protocols, complexity of web technologies, and network-layer insecurity. In website or web application, the client software usually cannot be controlled by the website or application owner. Therefore, input from the client software cannot be completely trusted and processed directly as an attacker can forge as a legitimate client, masquerade a user identity, create fraudulent message and cookies, or include links of malicious sites. Besides, HTTP is a session-less protocol. It is susceptible to replay and injection attacks. Messages in HTTP can easily be modified, spoofed and sniffed.

Because of the complexity of web technologies, conducting a detailed security analysis is not easy and straightforward. Therefore, website or web application should be designed properly to mitigate the security risks.

2.1 Website and Web Application Threats

Before discussing the security measure for the phases, it is important to have the understanding of the security threats associated with website and web application such that B/Ds can customise their security measures together with their needs. The corresponding counter measures will be discussed in depth later in Section 3.4 – Implementation.

The threats for website and web applications are:

¹ A collection of files accessed through a web address, covering a particular theme or subject, and managed by a particular person or organisation. Its opening page is called a home page. Websites typically use HTML to format and present information, and to provide navigational facilities that make it easy for a user to surf the site and around the world.

² A computer program designed to perform functions, tasks or activities for user using web as the user interfaces instead of traditional console-based interfaces.

Common Threats for Websites and Web Applications

- Using components with known vulnerability
- System/security misconfiguration
- Lack of defence in depth concept in design
- Distributed Denial of Service (DDoS) attack
- Altering website code and/or web defacement
- Web forgery

Additional Threats for Web Applications

- Injection
- Broken authentication and session management
- Insecure direct object reference
- Sensitive data exposure
- Missing function level access control
- Cross-Site Request Forgery (CSRF)
- Un-validated redirects and forwards
- Man-In-The-Middle (MITM) attack
- Cross-Site Scripting (XSS)
- Improper exception handling

2.1.1 Common Threats for Websites and Web Applications

In this section, each threat item will be discussed with examples to illustrate the potential impact if it successfully compromises websites.

a) Using components with known vulnerabilities

It is a common practice for a developer/programmer using pre-defined library, framework or modules by third party or community. Examples include:

- Unpatched components: Attacker can use known vulnerability to compromise the web server, to take the control and/or to suspend specific process such as web server process.
- Embedded components by vendor: Similar as unpatched components, if the website is built by embedded components but the vendor does not provide patches to customer, the component becomes a weakness and intruder may attack to this weakness.

- End of support software: If the website is running with end of support software, lack of official patch for fixing security flaw may cause a serious security exposure in future.

b) System/security misconfiguration

System/security misconfiguration affects the planned security measures deployed to website/web application. In general, the misconfiguration is caused by using the default setting by vendor without hardening, or residing developer access on production. Examples include:

- Default password: Attacker can use vendor provided default password to compromise the website for shutting down services in use.
- No access control: Attacker can read and write all files hosted in website.

c) Lack of defence in depth concept in design

Defence in depth is an important concept for protecting a computer environment with multiple layers of defensive measures. Even if the attacker breaks one of those defensive measures, other measures, such as responsive measures, will stop or delay the intrusion; Examples include:

- Running all components in a single machine: If the website contains several components, such as database or file repository, and they run on a single machine, the machine will become a single point of failure. When attacker compromises that machine, the attacker may take the full control of the website such that the information inside database and file repository will be leaked.
- Lack of component specific protection: Database, file repository, network device, operating system and web server application are the common components for a website. If the protection does not focus on each component, attacker may retrieve important information for hacking by compromising one of those components.

d) Distributed denial of service attack

DDoS attack is that an attacker launches the attack from a large number of controllable co-opted computers (known as botnet) to target sites or services with the aim at saturating all network bandwidth or server resources in order to disrupt the service to legitimate users. There are two major types of DDoS attack: volumetric and application layer attack.

- Volumetric attack: An attacker uses large volume of traffic to connect the website such that there is no bandwidth available for other users connecting to the website. For example, generating the workload of 100 gigabits per second (Gb/s) to the website with 100 Gb/s processing capability can halt the website services.
- Application layer attack: This attack overloads the application with unlimited service requests which may not be discovered easily as such attack uses slow traffic that appears legitimate in terms of the protocol rules and rates, and does not violate the security control policy.

e) Altering website code and/or web defacement

Web defacement is a threat by someone modifying the website without authorisation. One of the consequences is to cause embarrassment to B/Ds, another consequence is to insert malicious code for further attacks. Examples include

- Website replacement: The original website is replaced by hackers' website that may contain anti-Government information or illegal materials that affect to B/Ds' reputation.
- Trojan attached website: Attacker may attach malicious codes such as Trojan horse, virus and backdoor software to the vulnerable website. As a result, the web users may be infected by those attached malicious code such that B/Ds reputation will be affected.

f) Web forgery

Website forgery, or fake website, aims at masquerading a trustworthy (legitimated) website. It launches attack through other common communication channels, such as embedded website link in email, to divert users to access a malicious website that seems to be legitimate.

- Pretending a B/D's website: A fake website mimics the appearance of the victim's website including the style, layout and contents such that the fake website can deliver fake information to the public on behalf of the legitimated websites.
- Stealing classified information: If a fake website pretends with login function such as payment service by B/Ds, web users may not be aware of this fake website and input the real user credential to the fake website; this causes information leakage.

2.1.2 Additional Threats for Web Applications

In this section, each threat item will be discussed with examples to illustrate potential impact if it successfully compromises websites.

a) Injection

Injection is a security flaw by input of malicious code or invalid type of data using the authorised channels. The authorised channels are data fields for inputting name, date of birth, select query range, etc. Malicious code or invalid type of data uses the following ways for attacks: SQL injection of always-true logic (e.g. adding “or 1=1” in SQL query), input field injection (e.g. inputting characters instead of number for date of birth field), specific characters (e.g. !@#\$%^&*) and unknown command (e.g. “execute format c:”). By this method, attackers can trick the web application by performing unintended commands or retrieving data without proper access control such as disabling host-based firewall, retrieving database information and installing Trojan for targeted website / web application.

b) Broken authentication and session management

Authentication is one of the fundamental access controls for identifying authorised person and/or access. Session management allows authenticated user accessing the web application without re-login for a period. Improper controls will allow attackers to bypass the system without valid username/password, or to re-use authenticated session to pretend as an authorised user such that attackers can retrieve data or break web application’s security controls.

c) Insecure direct object reference

Direct object reference is commonly used for a programmer or developer indicating the linkage among different objects such as directory, file, database, and authentication servers. An insecure direction object reference makes the web application exposing extra information that is not necessary to the public or the web user. Example includes the filename and directory for web related files and the actual IP address of backend database.

d) Sensitive data exposure

Similar to “Insecure Direct Object Reference”, web application may expose classified data directly without proper data protection due to poor user authorisation and disclosure of data without authentication. The classified data may include Government classified information, personal information and authentication credentials.

e) Missing function level access control

Function level access control provides proper separation of access based on particular role e.g. user can view payroll information but cannot modify the payroll amount. If there is missing function level access control, web users (or attackers) will be able to forge requests without proper authorisation to achieve unexpected actions through web application.

f) Cross-site request forgery

CSRF is a security flaw that the victim's browser may send improper HTTP request to a vulnerable web application. For example, an attacker may force web user sending personal information to authenticate a public email website and log in accordingly. The attacker can then take controls of that authenticated website and perform illegal activities.

g) Un-validated redirects and forwards

Redirects and forwards are widely adopted for web application. Similar to the intended attack mentioned above, un-validated redirects and forwards may cause similar risk as XSS or CSRF such that user information can be exposed to attackers. Without proper validation, attackers can redirect victims to malware or phishing websites with authenticated information such as web cookie, which is a small piece of information stored in user side that may contain credential information.

h) Man-in-the-middle attack

MITM is a commonly used attack for hijacking data in transmission. Briefly, the attacker intercepts web connection and sniffs user information or classified data without acknowledgement of the web user. As this method does not affect web data transfer but additional copy of data is created, the web user may not be aware of this and he/she is being monitored by the attacker.

- Network traffic sniffing: an attacker can intercept the message to steal message by rebuilding the sniffed traffic.
- Replay attack: Together with traffic sniffing, an attacker resends a complete message that has been previously sent such that the attacker may gain protected message such as account balance information for authenticated person.
- Data leakage: Information transferred using website may be captured by attackers. If the information is classified and is not protected by proper security measure, it can be leaked to attackers.

i) Cross-site scripting

XSS flaw occurs when a web application takes untrusted data and sends it to a web browser without proper validation. An attacker can make use of this flaw to run malicious scripts in the victim's browser to retrieve sensitive information, deface websites, redirect the victim to untrusted websites, etc.

j) Improper exception handling

If a web application does not handle exceptions properly, it could be vulnerable to attacks. Examples include:

- Disclosure of sensitive information: The application displays more than enough information about the failure in the error messages. The messages may contain sensitive information which could supply additional knowledge for attackers to create an attack.
- Unauthorised access: If the application fails to an insecure mode, the attackers may bypass the security mechanisms to gain access rights to classified information which is normally inaccessible.

3. Website and Web Application Life Cycle

In Section 2, the security threats associated with website and web application have been discussed such that the reader should have the basic concept on the impact of the threats. This section would focus on the industry best practices for the SDLC of website and web application, which includes six phases as follows.

- Project Initiation
- Feasibility Study
- Systems Analysis & Design
- Implementation
- Post-Implementation
- Decommission

3.1 Project Initiation

A website or web application serves the B/Ds' specific purpose on providing service to the public or internal staff. The project initiation is the first phase for starting a development project of website/web application. In general, this phase is to gather project requirements such as functions and features of new website/web application, which are operation or business focused. Therefore, high-level security consideration can be defined for the next phase.

The information owner is the one who assigns data classification. The data classification is very important because it is one of the security requirements for the new project such as encryption requirements, data storage protection and physical protection requirement in adherence to Government security requirements. For example, if the website and web application handle CONFIDENTIAL information, several security requirements, such as encrypted storage and logical access control, shall be met.

In addition, as the project is not yet in implementation phase, B/Ds may designate a team to collect and analyse user requirements for next phase where the designated team can become the application development team. Therefore, one of the important tasks related to security is to understand the information classification involved to the website and web application.

3.2 Feasibility Study

A feasibility study is an assessment by B/Ds or authorised contractors for the practicality of the project. Practically for website and web application, the study should cover the security consideration based on the functional requirements in the project initiation phase. In security perspective, B/Ds should perform security requirement analysis and security planning if the project is technically feasible.

Security Requirement Analysis

The requirement analysis should cover the preliminary needs including system environment, functional and testing requirements. Examples include web server software, required encryption standard such as TLS and RSA 2048 bit, user access control such as administrator, read only user and testing through vulnerability assessment, penetration test and source code review. B/Ds should consider the operational feasibility and define the suitable requirements accordingly.

Security Planning

After the requirement analysis, B/Ds should have a high-level plan for fulfilling security requirements throughout the life cycle. In addition, a prototype or proof-of-concept (POC) exercise is suggested in this phase such that if it is restricted by some constraints (e.g. technical or resources), the project should be postponed or terminated. For the plans, they may include milestone and brief description on major activities instead of detailed items. It may also include areas such as configuration, security testing, authorisation and authentication, user training and security awareness plans. Examples include hardening requirement, security risk assessment and audit, tentative user privileged assignment, (security) skill transfer requirement and tentative audience for website and web application.

The plan should be broken down to actionable activities in later phases and being modified recursively until it is officially endorsed by stakeholders.

3.3 Systems Analysis & Design

System Analysis & Design (SA&D) is the core phase for development or acquisition of website and web application as the analysis result and confirmed design will be the input for development or selection criteria in acquisition. B/Ds may develop the website and web application using internal resources or external contractors, or purchase commercial website and web application. For acquisition, B/Ds may engage the solution providers to deploy websites and web applications (e.g. enterprise resources planning (ERP), payment system, portal website) and customise some of the components for fitting B/Ds' needs. For development, B/Ds may form an internal project team, including members from application development & maintenance team, LAN/system administrators, to build the website and web

application. From security perspective, B/Ds should perform the following activities including security requirement confirmation, security control design and security review.

Security Requirement Confirmation

This process is to confirm the security requirements based on the business needs, including the legal and regulatory requirements (e.g. the Personal Data (Privacy) Ordinance) and the Government security requirements. The requirements should align with current B/Ds' environment (e.g. security devices, system options), result by feasibility study and tentative system design. For example, if the website and web application handle classified information, encryption should be applied to suitable areas including the storage, transmission and processing.

Security Control Design

After confirmation of the security requirements, corresponding security controls should be incorporated to the design such that secure architecture, design and structure can be achieved. Examples include:

- Separation of development, testing and production environments – Development and testing environment should be separated from production environment as the production environment may handle classified information which is not allowed to be processed by other teams such as application development and maintenance team. Any patches or changes should also be verified in testing environment before applying to production environment.
- Use of testing data – Testing data should not contain any real information especially involving personal data and classified information. It should be generated with fake information.
- User access control – Least privileged and need to know are two security principles that related to user access control. In design phase, the system should consider the level of privileges such as read only access, or both write and read access. The access control should align with feature and function requirements defined in previous phase. Ideally, the control can be mapped to each function or functional group.
- Intrusion detection and log review – Intrusion detection is a widely adopted way to identify attacks. Log review is to find abnormal patterns performed by legitimated actions such as remote access, downloading file from website. In design phase, both should be considered as a control such that the corresponding log will be available when developing the web application, while the intrusion detection system (IDS) can be deployed as part of the website design such as host-based IDS and/or network-based IDS.
- Web application secure coding – The set of secure coding practices help application development team to withstand common web application threats discussed in Section 2.1.2 - Additional Threats for Web Applications of this practice guide. The secure coding practices will be discussed in detail in Section 3.4 – Implementation.

- Data integrity – Hashing technology can be used to assure data integrity. Papers analysing MD5 (Message Digest 5) have been published and revealed the weaknesses of MD5. Therefore, MD5 should not be used in new systems and MD5 in the existing systems should be replaced by stronger hashing algorithms, e.g. SHA-2.
- Temporary data handling – Temporary data resides within IT equipment such as servers, notebooks and printers can pose security risk of data leakage during unexpected circumstances. It is a best practice to delete temporary data at logoff or shutdown by enabling such built-in function of the equipment if applicable, or implementing secure deletion solutions. B/Ds should protect IT equipment based on the highest classification of information stored in the equipment including temporary data. To lower the physical security and other security requirements for the IT equipment, a common approach is to configure the device to prevent the storage of classified information on the storage media when in use and enforce deletion of temporary data at logoff or shutdown. It is important to note that there is no guarantee that the approach will always work effectively, therefore before implementing such a solution, B/Ds should consider the risk of residual temporary data that could arise from possible scenarios such as mis-configuration, loss of power, etc. To minimise residual temporary data, IT equipment can be configured to store temporary data in volatile memory such as Random Access Memory (RAM) disk if feasible. Furthermore, when flash memory devices such as flash drives, solid-state drives and hybrid hard drives are used, overwriting technique may not work effectively for temporary data removal due to the internal architecture in the flash memory devices. In such case, the media will retain its classification after the temporary data removal and the associated security requirements still apply in accordance with its data classification.

Personal Data Protection

When designing information systems containing personal data, appropriate technical and organisational security measures should be adopted to protect personal data from unauthorised or accidental access, processing, erasure or other use, including but not limited to ensuring the compliance with all applicable laws and regulations, conducting privacy impact assessment to identify and manage data protection risks, ensuring the processes and systems are designed such that the collection and processing of personal data are limited to what is necessary for the identified purpose, and improving staff awareness of the possible consequences (such as violation of security policies, damage of government image, disciplinary actions) when personal data is breached.

For better protection of personal data in the information systems, B/Ds should observe the following guidelines as developed by PCPD.

- Guide To Data Protection by Design for ICT Systems
(https://www.pcpd.org.hk/tc_chi/resources_centre/publications/files/Guide_to_DPbD4ICTSystems_May2019.pdf)
- Privacy Management Programme
(<https://www.pcpd.org.hk/pmp/pmp.html>)

Moreover, when designing web applications that may involve personal data, the following guidelines should be considered:

- Limit the collection of personal data to the minimum that is relevant, and necessary for the identified purposes.
- Limit the processing of personal data to the extent that is adequate, relevant and necessary for the identified purposes.
- Minimise the exposure of personal data by applying anonymisation techniques (e.g. removing or masking the identity of individuals).
- Ensure that the personal data is erased when no longer necessary.

Security Review

The concept of security by design is important for identifying the potential threats for website and web application and taking appropriate remediation before development / acquisition. All often, if the security requirements are defined properly and the identified risks are addressed in the early stage, the rework effort could be immensely reduced. B/Ds are recommended to conduct security review in the design stage in the SDLC, which serves as a checkpoint to ensure necessary security requirements are identified and incorporated in the system design

The review should assess the adequacy of the security requirements and review the system design by identifying possible compliance issues as well as the security risks with reference to the website and web application threats discussed in Section 2, in addition to other infrastructural threats in the operating environment (e.g. insecure network segregation). After performing the review, the identified risks and recommendations should be documented and addressed in the design stage or other phases appropriately. It is a best practice to include a role in the development team for assessing security risks, proposing potential security-related issues, and performing security reviews of the system design and programming code. The pre-production Security Risk Assessment (SRA) will verify the follow-up actions for the security review and the quality of the programming code review to ensure necessary security measures and controls are implemented in the system properly.

3.4 Implementation

Implementation phase includes three major parts, system/application development, security assurance and system installation & deployment. For system/application development, the application development team (or the external contractor) should follow the requirements defined and the suggested counter measures from risk assessment result. For security assurance, B/Ds should exercise due diligence to ensure that the website or web application is ready for production with acceptable risks. For installation & deployment, B/Ds should perform inspection and acceptance of developed components for website and web application and integration of security controls to current environment, or new production environment, if any.

3.4.1 System/Application Development

In this section, we are going to discuss the counter measures to address the security threats associated with websites and web applications, which are described in Section 2.

Counter Measures for Common Threats of Websites and Web Applications

- Using components with known vulnerability
B/Ds should apply latest patches for all related components after proper change management and perform regular security risk assessment to identify known vulnerability. B/Ds may utilise the web vulnerability scanning facilities established by GovCERT.HK to conduct scanning on their Internet-facing websites to examine the security status. Also, B/D may need to deploy web server health monitoring utilities to gain visibility into the security status and performance, identify configuration changes to improve performance or address operational challenges, and maintain a highly optimised and top performing environment.

If there is known vulnerability, B/Ds should follow the change and patch management and fix the issue as soon as possible. In the worst scenario, B/Ds may consider to suspend the web application temporarily if the impact is critical to B/Ds. Examples include Heartbleed vulnerability in 2014. This vulnerability allows an attacker to steal private keys, user's session and password which would cause data leakage including classified information.

- System/security misconfiguration
The countermeasures are defining the hardening requirements with baseline, regular scanning and auditing of the website to ensure that the configuration is in good standard and applying tested security patches after proper change management process.
- Lack of defence concept in design
Implement multiple tiers design for network segregation as described in Annex A - Website Security Architecture.
- Distributed denial of service attack
Implement multiple tiers design for network segregation as described in Annex A - Website Security Architecture.
- Altering website code and/or web defacement
Tighten the end-point servers' security as described in Annex B – Web Server Security.
- Web forgery
Assure authenticity of the website as described in Annex C – Secure Website with HTTPS Protocol.

Counter Measures for Additional Threats of Web Applications

- **Injection**
A safe application programming interface (API) is recommended to avoid malicious code injection, ideally, the input fields should use pre-defined parameter instead of free text value such that attacker cannot inject malicious code through the web application. In addition, if free text value is not avoidable, the programmer should apply a white list input validation such that only expected value can be processed.
- **Broken authentication and session management**
Ideally, there should be a set of authentication and session management for a web application. Authentication should be enforced for all web application components unless the information is intended to the public. A time-out or re-authentication mechanism should be in place that align with operation requirements. For example, if the web application is handling classified information, programmer should configure a shorter inactive timeout. In addition, if there is any indication in the change of the session's originator, e.g. change in source IP address, re-authentication is recommended.
- **Insecure direct object reference**
For direct object reference, additional layer of defence such as user authentication or trusted list should be applied. In addition, it is preferable to use indirect object reference such that the object reference is generated based on an authentication user or session which attacker cannot re-use the indirect object reference without proper user/session information.
- **Sensitive data exposure**
For B/Ds, sensitive data should be treated as classified information by Government terminology. Program should avoid storing classified information if it can be avoided. If there is a need for storing classified information temporarily, program should discard the information whenever it is no longer required. Adequate access control should be applied to the folders and files storing classified information in the web server to ensure that users cannot access any files not intended for user access. In addition, encryption should be applied according to Government security requirements. Information confidentiality can be assured by providing bi-directional encryption of communications as described in Annex C – Secure Website with HTTPS Protocol which protects against capturing unencrypted sensitive data during data transmission over the Internet. Autocomplete feature should be disabled if the input form may contain classified information. B/D should take cautions to protect unreleased classified information stored in the web server by proper access control setting and better generating randomised text for web links.
- **Missing function level access control**
Function level access represents the access right for different functions in web application. Examples include user access modification, user information listing function. Programmer should have a clear view and requirements of access right for each function. Ideally, all functions should be denied by default and only authorised user roles can access particular functions. In

addition, it is preferable to hide the unauthorised function in the presentation layer (e.g. web user interface) such that the web user cannot activate the functions directly through the web browser.

- **Cross-site request forgery**
The threat relates to HTTP request by unauthorised person. For example, if you log to a social media website and simultaneously accessing another website, attacker can forge HTTP request on behalf of your authorised information. Similar to B/Ds' websites, the countermeasure is performing proof for intended access by using Completely Automated Public Turing test to tell Computers and Humans Apart (CAPTCHA), which shows that the login is initiated by a real person rather than automated by malicious code. Alternatively, programmer may include a unique token in a hidden field such that the HTTP request becomes invalid as missing the unique token.
- **Un-validated redirects and forwards**
Redirection and forward is a feature in web application, however, if the action is not validated, it may cause security flaw. The countermeasure for this threat are avoiding redirection and forward where possible. If it is necessary to use redirection feature due to user requirements, proper checking should be adopted such as user authorisation and input value validation.
- **Man-in-the-middle attack**
Apply bi-directional encryption (e.g. TLS) when exchanging information between web users and web server as described in Annex C – Secure Website with HTTPS Protocol which protects against tampering with or forging the contents of the communication.
- **Cross-site scripting**
Input validation would help the programmer to avoid known pattern of XSS type code. In addition, Content Security Policy (CSP) is supported by latest web browser which is able to against XSS. It is offered by World Wide Web Consortium (W3C), which is an international standard organisation for the World Wide Web. Programmer should consider adopting this when developing the web application.
- **Improper exception handling**
The “fail securely” principle should be observed if application failure occurs. For example, when an exception occurs, the application should provide meaningful error message that is helpful to the user or the support staff yet ensure that no sensitive information will be disclosed. Another example is to design the security mechanisms to deny access by default so that attackers cannot gain privileged access to classified information during a failure.

Web Application Secure Coding

On top of the counter measures discussed above, B/Ds should also observe the web application secure coding practices in developing web applications so as to withstand some common web application security vulnerabilities. B/Ds can refer to Annex D – Web Application Secure Coding Practices for the detailed practices.

In addition, secure coding practices and checklist from the Open Web Application Security Project (OWASP) are available in OWASP official website. It is recommended that B/Ds take reference to these coding practices. A customised version is presented in Annex E – OWASP Secure Coding Practices Quick Reference that provides brief descriptions for 14 areas suggested in these coding practices.

3.4.2 Security Assurance

Security assurance is an important step for website and web application as it provides a certain level of confidence and verifies the residual risk level. In general, B/Ds should perform security test, source code review for web application, and security risk assessment to ensure that the website or web application is ready for production with acceptable risk.

Security Test

Before a website or web application is launched for production, the need for comprehensive testing is paramount. In addition to user acceptance tests, there are others, such as system tests, stress tests, regression tests and unit tests that are useful in validating the performance and accuracy of system functionalities. This section describes some of the tests that can be carried out in order to increase the reliability and security of the program/systems being developed.

- Security configuration review – It is a review exercise to confirm that the security configuration of the IT equipment in the pre-production environment is properly set as suggested in the previous phase. Proper configuration should refer to the hardening guides and the confirmed configuration requirements in the development phase.
- Unit Testing – Unit testing is an important part of the development stage, designed to identify the vulnerabilities in a website or web application. Unit testing involves the testing of individual modules to ensure that all internal operations of a module perform according to specifications. Unit testing should include tests for common security issues, such as buffer overflows, and is especially important if the module is being integrated into a “build” with other components. If no unit tests are carried out, it becomes very hard to implement an automated security testing process in the middle of the development stage.

Source Code Review and Protection

Source code review is a white box approach for identifying vulnerability or security flaw by detailed comparison between designed logic and actual coding. It is possible that the programmer/developer inserts malicious code to the web application such that it is beneficial to himself/herself. Examples include copying classified information in unencrypted format and sending to him/her. Generally, the source code review can be performed by manual review or automated review using commercial tools. Automated review is also referred as source code security scan. B/Ds should adopt the source code review and select the suitable way.

- Manual review – it is a labour intensive review process by peer-review or independent third party with expertise in particular programming language. For example, if the web application is developed by language A, it is expected to have an expert in language A who is capable to identify the security flaw. B/Ds' staff who are not involved in the development cycle or external contractors can perform this review for independent consideration. However, manual review is a labour intensive activity such that B/Ds should select important modules or components for review if there are limited resources.
- Automated review by tools – there are many commercial products providing automated source code review based on particular programming languages. The tools examine the code and find common errors that lead to security bugs. They alert developers and provide suggestions for fixing. They are capable to identify common risks mentioned in previous sections, and it works faster than manual review. However, since the automated review tools are based on known programming pattern or logic for detection, it is possible that some experts hide the malicious code. In addition, manual verification is recommended for the security flaw detected by tools in order to avoid false positives. Therefore, when using automated review tools, B/Ds should consider adopt manual review as the second opinion of the detected security flaws.

Source code protection is important to protect the intellectual property, and to avoid attacks as the code contains valuable information to hackers/attackers.

- Disable right-click function – for some popular web browser, user may right click the website to view its source code. The code may contain important information that helps attackers to launch an attack. Although there are many tools available for viewing the code, it is still recommended to adopt this control if right click function does not affect web application's function.
- Code Signing – it is a mechanism to verify that the program code has not been altered. Briefly, a hashed code is generated by trusted party such as Certificate Authority, then the browser will execute the web application with the valid code sign. However, it is programming language dependent such that B/Ds should adopt this measure where possible.
- Obfuscation – it is a technique to introduce the confusion of reading the code and making the hacker harder to understand the logic behind the code. It aims at protecting the code against reverse engineering and hacking attempts as the code is presented in a way that is difficult to be read by human. B/Ds should use obfuscation tool to protect the source code and hide the application details as far as possible.

Security Risk Assessment

A security risk assessment should be performed before the production launch of the website or web application. The objective of the risk assessment is to ensure that the risk level is acceptable by B/Ds' management. B/Ds should conduct vulnerability scan, penetration test and source code security scan for all Internet facing websites

and web applications regularly, before production, and prior to major enhancements and changes associated with websites or web applications. The identified vulnerabilities and issues should be evaluated and addressed with appropriate corrective actions before system live run or production.

Privacy Impact Assessment

For web applications having potential privacy implications, privacy impact assessment should be conducted to examine the adequacy, effectiveness and practicability of the planned protection measures for the protection of personal data. B/Ds may make reference to the leaflet on privacy impact assessment published by the Privacy Commissioner for Personal Data, which is available at PCPDs' web site:

https://www.pcpd.org.hk/english/publications/files/PIAleaflet_e.pdf

In relation to personal data protection, B/Ds shall further refer to the mandatory requirements as specified in Data Protection Principles of the Personal Data (Privacy) Ordinance which is available at https://www.pcpd.org.hk/english/data_privacy_law/6_data_protection_principles/principles.html

3.4.3 System Installation and Deployment

After completing the SRAA and applying the remediation, the website or web application should be attained to an acceptable level of residual risk. In this phase, B/Ds should apply suitable controls for production environment. In general, a pre-production environment should be prepared with positive result of system integration test (SIT) and user acceptance test (UAT). In this stage, B/Ds should ensure that system installation and deployment activities are as planned, including the rectification from the findings of security assurance tasks. Ideally, a post-implementation review should be performed right after the website or web application is in service to the public and/or Government staff. The post-implementation review will be discussed in the coming section.

3.5 Post-Implementation

It is the phase of on-going operation and maintenance for the website or web application. In this phase, the security consideration focuses on configuration/change management, continuous monitoring and the security incident handling. List of security considerations are shown below:

Post-Implementation Review

Right after the website or web application is in production, a post-implementation review should be performed such that there is no unexpected problem during the system installation and deployment task. Ideally, the review should include two components – document review and sample test.

- Document review – B/Ds should review the process of installation and deployment task based on the records (documents) prepared. The documents should include the major activities and B/Ds can check if there is any violation or missing tasks against the planned security requirements.
- Sample test – B/Ds should perform SIT, UAT and security assurance by sampling as a spot test to verify the effectiveness and completeness of security practice after the document review. The sampling should depend on the criticality of the website or web application and the resources available by B/Ds.

B/Ds may adopt other review items for post-implementation review as defined in SA&D or implementation phase. The key objective of these review items are to confirm the security and quality against the production environment and planned requirements.

Change and Patch Management

Change and patch management are important for website and web application to maintain a certain level of security with acceptable risk. All proposed changes or patches should be verified in testing environment and they do not incur security issue to the production environment. Several components are suggested as below:

- **Change Committee** – Ideally, a change committee should be set up for approving the suggested change due to feature enhancement or bug fixing. However, if the website and web application is not justified to setup a dedicated committee by B/Ds' management judgement, B/Ds should rely on existing departmental change and patch procedures to maintain the level of confidence in applying changes/patches.
- **Change/Patch Plan** – A plan is important for every change/patch. The plan should include the detailed procedures customised for particular website and web application. The plan may include testing procedure in testing environment, deployment procedure for production environment and fallback procedure if the tested change/patch does not work in the production environment. In general, backup should be performed in production environment before there is any change/patch applied.

Backup

Backup is important for the on-going operation especially when there is an unexpected loss of information, the information can be retrieved from backup. However, if the website and web application handle classified information, the backup should fulfil the Government security requirements by adopting necessary protection. Together with change and patch management, additional backup task should be performed to ensure that fallback is available when the change is failed.

Security Monitoring

Similar to other information systems, security monitoring is important to detect malicious activities targeted to website and web application. As discussed in previous section, several security measures are considered for website and web application, including firewall, NIPS/NIDS, Web Application Firewall (WAF), anti-DDoS solution, access logging by web application. Several components are suggested to B/Ds for consideration:

- **Centralised monitoring platform** – A centralised monitoring platform provides a single view of security issue detected by those security measures. In addition, the maintenance team and incident response team may use this centralised platform to correlate different information sources for better visibility of attacks targeted to website and web application.

- Real time alerting mechanism – Incident response time is an important measurement to react and to reduce the impact of an attack. Real time alerts provide promptly warning to incident response team and/or maintenance team to react to the detected issue and apply suitable countermeasure(s) as soon as possible.

Incident Response and Handling

An IT security incident is any adverse event that could pose a threat to the availability, integrity and confidentiality of an information system or information asset. For website and web application, the example of security incidents include malicious code attacks, unauthorised access or utilisation of services, denial of web services, compromise of web user privileges, SQL injection for stealing classified information.

In general, an incident response and handling procedure should be defined for website and web application. If existing incident response and handling procedure is applicable to newly developed website and web application, B/Ds should consider revising the scope of existing process and procedure such that the new system is included for on-going operation. If B/Ds consider the new system is important but the existing process and procedure do not cover the new system well, B/Ds should design a new incident response and handling procedure accordingly.

In addition, B/Ds should refer to the Practice Guide for Information Security Incident Handling and the defined departmental incident handling procedure if available.

Security Risk Assessment and Audit

Security risk assessments for websites and web applications shall be performed at least once every two years or after major changes. In addition, security audits on websites and web applications shall be performed periodically by independent auditors to ensure the compliance of IT security policies and effective implementation of security measures.

3.6 Decommission

When it is confirmed to decommission a website and web application, B/Ds should perform following activities such that the information handled by website and web application will not be leaked to unauthorised parties. The activities are:

Replacement for decommissioned website or web application

If there is a replacement of website or web application for the original system, B/Ds should ensure that the replacement is run properly and necessary migration has been performed such as data and user migration. If there is no replacement, B/Ds may perform the three activities listed as below.

Information Preservation

For website and web application, regardless information classification, the informant may be useful for other application or archiving is necessary for B/Ds operation. Therefore, B/Ds should confirm that the information is required to be retained or removed without any archive. If archive is required for classified information, proper protection should be applied. Alternatively, if archive is not necessary, B/Ds should perform the media sanitisation for all backups and storage in use for website and web application.

Media Sanitisation

Media sanitisation is applicable for all backup copies and storage in use. B/Ds shall fulfil the Government security requirements if backup media is storing classified information from the website and web application. B/Ds should identify the scope of media that stores classified information in production environment. If the media is sharing with other systems (i.e. shared platform), B/Ds should perform risk assessment to identify the potential risk and ensure that the same level of protection are in place. Examples include:

- Protection for Shared Platform – Since the website and web application may be hosted at a shared platform such as web server clusters, file servers and database server clusters. If the overall classification of shared platform is degraded due to decommission, for example, the classification degraded from CONFIDENTIAL to RESTRICT, B/Ds should consider the residue risk of data leakage if security controls are downgraded. If there is any doubt, B/Ds should retain the security controls as if.

Hardware and Software Re-use or Disposal

At the end of decommission phase, there may be residue hardware and software that can be re-used or disposed. Depending on the situation, from both environmental protection and information security perspective, if there is acceptable risk for re-using hardware and software, it is recommended to do so. Before hardware re-use, B/Ds should consider cleaning up all program and data securely especially if classified information was in use before. For software, if the license is applicable for re-use in other servers, B/Ds should consider installing the software to new servers instead of existing decommission environment. As the existing environment may contain information that is not desirable for disclosing to other, even if it is not classified information. If it is confirmed to dispose or re-use the hardware and software, B/Ds shall follow Government security requirements for the destruction of classified information. All classified information must be completely cleared from media before disposal, or re-use. Any method that only temporarily erases the classified information or allows alternative means of recovery must not be used. If the classified information cannot be completely cleared, the media unit must be physically destroyed in a manner that prevents recovery of the classified information.

4. Additional Security Consideration for Web Content Management System and Web 2.0

Apart from the security risks and counter measures for website and web application discussed, there are additional security considerations when implementing particular type of web application such as Web Content Management System (WCMS) and Web 2.0 technology.

WCMS is a website management tool that facilitates a group of users, usually from different departments in an enterprise, to collaboratively maintain and organise the contents of a website in an effective manner. WCMS has grown in importance as more and more organisations communicate and publish their information via the web. Like other web-based applications, WCMS' applications are exposed to the same set of common security threats found in any network and web-based operation or process. However, there are several additional counter measures specifically for WCMS. In Section 4.1, details of security threats and counter measures on WCMS will be discussed.

Similar to WCMS, Web 2.0 is a set of web applications and websites that provide collaborative services to users. While Web 2.0 technologies offer many advantages in terms of enriching the Internet and improving the user experience, they also bring a number of security concerns and attack vectors into existence. Since one characteristic for a Web 2.0 application is to encourage greater user involvement, the exposure of the individual user or client to security threats and vulnerabilities increases. In Section 4.2, details of security threats and counter measures on Web 2.0 will be discussed.

4.1 Web Content Management System

WCMS is an application built on top of existing web technology. Like other web applications, a WCMS is subject to the same security threats and operation process vulnerabilities as other web applications. In this section, we discuss the common security concerns and ways they can be mitigated.

4.1.1 Security Concerns

Given that a WCMS is a software application, it is prone to bugs just like any other programs. Vulnerabilities have been found in WCMS. As one example, a vulnerability called “absolute path traversal vulnerability” was found in the open source product OpenCms in 2006. This flaw would allow remote authenticated users to download arbitrary files. Another security concern lies with protection of authentication credentials when accessing a WCMS. Many WCMS products are designed primarily to solve the contents management problem of websites rather

than building a secure product. Some WCMS products do not provide adequate protection for logins and passwords, for example, these passwords, including the administrator password, are sent as plain text over the network.

Similarly, as part of the publishing/uploading process, a WCMS might use file transfer protocols such as FTP to transfer files from the WCMS data storage server to the web server. FTP is not a secure protocol in the sense that authentication credentials and passwords are sent as plain text over the network. In addition, because publishing is an automatic process from the WCMS to the production web server, FTP credentials might be hard-coded in certain configuration files. Usually a hard-coded login password like this will not be changed regularly. As a result, any leakage of this password could allow someone to access to the web contents on the production web server illegally.

If the WCMS includes other modules, individual subsystems may have their own bugs and introduce their own vulnerabilities to the WCMS. For example, if the WCMS has an email module, it might be prone to the same common threats faced by email server such as email spoofing. On top of this, the backend database server of the WCMS may have its own vulnerabilities as well.

4.1.2 Precautionary Measures

There are a number of precautionary measures that should be done proactively to mitigate the security threats identified above:

- Follow best practices by applying the latest security patches to all web server software. Any alerts or warnings about vulnerabilities on the WCMS product being used should be addressed immediately, especially if the WCMS can be accessed directly from the Internet. Any patch management process should also address additional WCMS modules, including email subsystems, backend database servers, JAVA runtime environments, and so on;
- A strict password policy should be defined. This should include a minimum password length, initial assignments to personnel, restricted words and formats, and a limited password life cycle;
- Logins and passwords sent over the Internet should be protected by TLS, so that attackers cannot sniff them over the network. In general, access to administration pages should be further controlled and these should not be open to Internet access;
- When publishing any web contents from the WCMS to the production web server, file transfer programs such as FTP should be replaced by a Secure Shell (or SSH) that protects transmission channels by encrypting data. Some SSH implementations also support a feature that controls which IP addresses are allowed to connect to the destination server;

- To enforce data security, many WCMS implementations have built-in access control whereby groups of users are segregated into editor and administrator (approver) roles. These roles and their corresponding access rights should be clearly defined and reviewed periodically; and
- A good WCMS should keep an audit trail, logging all editing and approval activities. These audit trails should be retained for a period commensurate with their usefulness, and should be secured so they cannot be modified and can only be read by authorised persons.

4.2 Web 2.0

“Web 2.0” does not have a precise definition. To many people, the phrase refers to special web application technologies and websites, such as weblogs and wikis, which use the Internet in a collaborative way to provide services to users. Web 2.0 relies in large part on the user-as-publisher model of interaction and allows for user-created contents to be developed and implemented by large groups of individuals. These technologies are increasingly being used by companies for better staff collaboration and communication. O’Reilly outlined seven principles that can help to distinguish the core features of Web 2.0 applications¹.

4.2.1 Security Concerns

While Web 2.0 technologies offer many advantages in terms of enriching the Internet and improving the user experience, they also bring a number of security concerns and attack vectors into existence. Because one characteristic for a Web 2.0 application is to encourage greater user involvement, the exposure of the individual user or client to security threats and vulnerabilities increases. The following are common threats in the Web 2.0 arena.

a) Security Threats Related to Asynchronous JavaScript and XML (AJAX)

In order to provide a rich user experience, many Web 2.0 sites have employed lightweight user interface code such as AJAX. In the traditional client-server models, the majority of requests are handled and processed on the server side. AJAX allows a higher proportion of requests to be processed on the client side. This may give malicious users greater opportunity to modify any application code running on a client computer when probing and testing an application for vulnerabilities. As AJAX can be used in conjunction with a large number of web services, by enabling connectivity between them, this could present additional attack vectors into which malicious users could inject hostile content. As an example, AJAX could serve to amplify the potential of XSS attacks, which seek to inject code into legitimate websites in order to mislead users and steal their information. Not only would this

¹ <http://www.oreilly.com/pub/a/oreilly/tim/news/2005/09/30/what-is-web-20.html>

allow an attacker to steal sensitive information, it could also allow an attacker to insert malicious code onto the host through malicious scripts.

One security vendor has categorised a new class of vulnerability as JavaScript Hijacking. This class of vulnerability specifically affects Web 2.0 AJAX-style web applications. Through this vulnerability, an unauthorised party can read sensitive data contained in JavaScript messages. An application may be vulnerable if JavaScript is used as a data transfer format, and in particular, when sensitive or classified information is being handled.

b) Security Threats Related to Web Feeds

With the characteristic of decentralised and distributed contents for Web 2.0, web information is distributed to other sites via lightweight syndication protocols, such as RSS and Atom. These web feeds allow both users and websites to obtain content headlines and body text without visiting the site in question. There is no standard mechanism to authenticate the publishers of feed entries. As such, malicious attackers can make use of these web feeds to inject literal JavaScript into the RSS feeds to generate attacks on the client browser. All that the attacker needs to do is to insert a literal script injection into standard RSS or Atom elements, such as the Title, Link or Description XML tags for RSS. When an end-user visits this particular website and loads the page with the RSS feed, the malicious script will be executed.

c) Data Privacy and Intellectual Property Rights

It is a further point that should be noted with regard to user concerns about privacy and rights to protect their own data. In many of the early Web 2.0 applications, copyright was only loosely enforced. For example, Amazon lays claim to all and any reviews submitted to the site, but in the absence of enforcement, people may repost the same review elsewhere. However, as organisations begin to realise that controls over data may be their chief competitive advantage, there will be greater attempts to control access or distribution of data.

d) Impact on Internet Resources

The wide adoption of AJAX may also impact the network. The use of AJAX technology can result in frequent (non-user-triggered) or even constant data exchanges between a client and a server, and any excessive delay or data loss during these data transfers may have effects that are visible to the users. While it is possible that AJAX (and AJAX developers) will evolve to suit the network (for example, handling the delay or loss in the background), users may also demand more consistent and reliable network performance than today's Internet can deliver. Currently, any request for consistent network performance is usually met by IP QoS mechanisms, but implementing QoS on the public Internet will be considerably more challenging than on a private intranet. Certainly, other solutions may surface, making this an interesting area to watch for in the future.

4.2.2 Precautionary Measures

The security concerns discussed in the previous section need to be addressed with necessary precautionary measures. Below we list a small selection.

a) Security through Controls

As mentioned, Web 2.0 applications are highly client centric. This approach may pose significant threats to a system if adequate controls are not in place. To build an interactive and secure Web 2.0 application, a secure architecture with appropriate controls is an essential component. Some of the building blocks of this architecture include:

- A solid session management scheme to ensure that authentication and authorisation is performed inside a trusted part of architecture;
- Data validation is performed in both directions on the server-side at various layers to limit or prevent injection and other forms of attacks; and
- All calls to backend services are performed by trusted server-side business logic.

b) Security through Openness

Given that open-source software or APIs are exposed to open scrutiny, they are usually developed with security in mind. Hence, they generally have increased security built-in. Instead of following a proprietary approach, proven security protocols and industry standards should be used. If open source software or APIs are used, the software should be tracked to ensure that all licences are valid for use, and published vulnerabilities from these open source software solutions should be addressed in time.

c) Corporate Governance on Web 2.0

Although current Web 2.0 services are mostly public services, outside the organisation, management still needs to be aware of the risks that may impact corporate members who have access to these services. Policies should be established to protect sensitive corporate and/or customer information, and ensure this will not be disclosed in open websites such as blogs. Regular awareness training should also be conducted to educate staff about the company's IT Security Policy and strengthen security awareness around the risks associated with these new technologies.

To avoid the risks associated with web feeds, only data feeds from reputable sources should be trusted. For application developers who provide web feeds, preventive measures such as white-listing only those necessary HTML tags should be deployed. This can reduce the possibility of XSS attacks on web feeds. The following are additional best practices that IT practitioners should consider:

- Although wikis can lead to broader and more rapidly evolving coverage of topics, they are vulnerable to misinformation and anonymous authors could make malicious or unauthorised changes to information being published. In case a wiki-type application is to be deployed, editorial controls should be imposed to restrict updates to only legitimate and authorised areas. Proper authentication and access control should also be imposed to better ensure the integrity of contents.
- When a blog is used to communicate an organisation's vision, or for other promotional purposes, care must be taken to avoid possible leakage of sensitive or proprietary information. Monitoring and filtering of all blog contents should be implemented. Acceptable use of policies for blogs should also be distributed to all users.
- Like other applications, Web 2.0 programs should undergo vigorous vulnerability testing to identify loopholes and uncover any weaknesses, including command injection, XSS and buffer overflow vulnerabilities. All problem areas should then be fixed and security threats be mitigated before the application is released into the production environment. In addition, periodic security assessments should be conducted on a regular basis.

d) End User Awareness

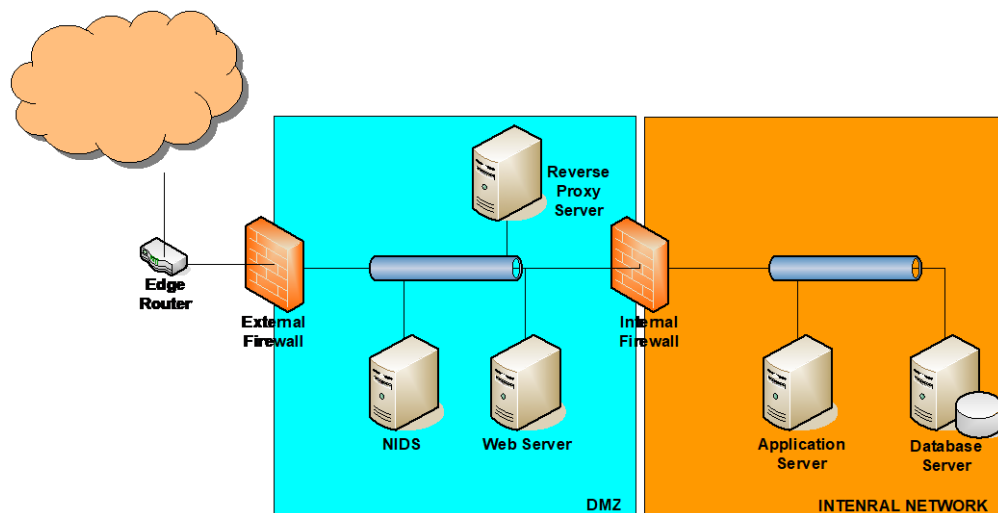
Other than the security measures for web server, protecting end user machines is another important area. To avoid a computer from being compromised and becoming a weapon to attack other machines, web application and the Internet users are advised to:

- ensure that the operating system and key system components such as the web browser is fully patched and up to date;
- install a personal firewall along with anti-malware tools with the latest malware definitions that can detect malware such as keyloggers;
- employ different sets of login and password combinations for different web applications and services;
- regularly change passwords in critical web applications if a one-time password system is not supported; and
- turn off all JavaScript or ActiveX support in the web browser before visiting any unfamiliar websites.

*** ENDS ***

Annex A: Website and Web Application Security Architecture

A typical website and web application architecture contains three tiers, separating an external facing web server, application server, and database server as shown in the diagram below. With such a tier-based architecture, even if an attacker compromises the external facing web server from outside, the attacker still has to find ways to attack the internal network.



The external facing web server should be confined within a demilitarised zone (DMZ) which is a special network segment containing servers with access to Internet services. Servers with sensitive information are located in the internal network with additional protection. The internal and external firewalls should be from different vendors or types so that the firewalls will not have the same vulnerability. For example, the external firewall can be a web application firewall while the internal firewall can be a network layer stateful inspection firewall.

Network intrusion detection system (NIDS)/intrusion prevention system (NIPS) should be installed to detect/prevent attacks or suspicious traffic in the DMZ. Alerts and reports from the NIDS/NIPS should be actively reviewed to identify attacks at the earliest possible moments. In addition, NIDS/NIPS should always be updated with latest attack signatures provided by the vendor. In addition, Web Application Firewall (WAF) and anti-DDoS are application specific security devices to protect a website and web application against common threats such as injection and application-based DDoS attack. Therefore, they should be considered to be deployed for the monitoring and blocking of web traffic in the DMZ.

Anti-DDoS Protection

The possible security control to address the threat of DDoS attack at network gateway level is anti-DDoS protection which will be further elaborated as follows:

a) Function

In general, DDoS attack is initiated from Internet. As such, an effective way to address both the volumetric and application layer DDoS attack is to adopt the anti-DDoS clean pipe solution. This anti-DDoS service provides real-time protection to analyse the network traffic to block malicious traffic and permit the legitimate traffic. The data traffic would firstly pass through a service provider's scrubbing centre where malicious traffics are filtered and legitimate traffics are then redirected to enter into an organisation's network. A scrubbing centre usually provides protection services on volumetric congestion with bandwidth over 1 Tera bit per second, application-layer, automatic blocking of known attack signatures and 7 x 24 security operation centre support. This solution offers benefits on real-time and proactive mitigation on DDoS attack, better bandwidth utilisation with network-based defence as well as the immediate notification on the attack event.

b) Implementation considerations

B/Ds should define the service requirement such as the service level agreement (SLA), the subscribed clean pipe bandwidth and the business needs for the anti-DDoS attack protection in order to engage anti-DDoS outsourcing or external service provider. Clean pipe bandwidth is one of the important factors to be considered. When attack traffic volume exceeds the subscribed clean pipe bandwidth, it will lead to black hole of traffic. Thus, B/Ds should evaluate the clean pipe bandwidth requirement and the service options (such as acquire additional bandwidth on demand) offered by the anti-DDoS outsourcing or external service provider.

The other important factor to be considered is whether HTTPS encrypted traffic will be monitored and analysed by the anti-DDoS outsourcing or external service provider. If this is the case, B/Ds have to authorise the anti-DDoS outsourcing or external service provider to use the Government's digital certificate. B/Ds should use contractual terms to limit the anti-DDoS outsourcing or external service provider to use the Government's digital certificate only when necessary as part of the service delivery. On the other hand, B/Ds have to require the anti-DDoS outsourcing or external service provider to sign non-disclosure agreement and offer the rights to B/Ds to conduct security audit to ensure proper security measures are in place to protect B/Ds' traffic being processed by the outsourcing or external service provider.

c) Operational considerations

• User

Anti-DDoS solution should be transparent to user and user's involvement in operation is minimal. User may receive alert notification and routine reports about the DDoS attack and service level summary. User should raise enquiry or report to IT staff when there is potential access issue, such as inaccessible for website or missing email.

- IT Staff
IT staff is responsible for administration of the anti-DDoS solution provided by outsourcing or external service provider. IT staff is required to observe and follow up timely for any DDoS attack alerts issued by anti-DDoS outsourcing or external service provider to ensure the Internet network service and related e-services are sustained. In case there is interruption of service resulted from the black hole of traffic, IT staff should follow the security incident handling procedure to handle the security incident until it is mitigated. In addition, IT staff should review the service report and follow up issues with anti-DDoS outsourcing or external service provider such as missing SLA and unresolved technical problem, and recommend the subscription of additional bandwidth where appropriate

Annex B: Web Server Security

The following are suggestions to B/Ds when performing system hardening for associated servers such as web, application, database and file.

- Follow security requirements based on the information classification;
- Avoid using same set of web servers handling different levels of classified information such as CONFIDENTIAL and RESTRICTED. If it cannot be avoided, B/Ds shall use the highest level for applying security controls;
- Develop system and application hardening guideline taking reference by vendor's security guidelines;
- Configure the servers according to hardening guideline;
- Run web server processes with appropriate privilege account. Avoid running the web server processes using privileged accounts (e.g. 'root', 'SYSTEM', 'Administrator');
- Apply latest approved security patches to the website components;
- Avoid using the end of supported components unless there is no alternative;
- Configure access rights strictly according to system hardening guideline and application requirements such as read-only access for information to public;
- Disable all unused accounts, including user and default accounts. If possible, the unused accounts should be removed if there is no impact for running website and web application;
- Avoid storing users' passwords in database or file without proper protection such as hashing and/or encryption;
- Analyse access log to monitor malicious activities and support incident investigation, including access to files, links from requestors as well as access to internal resources such as database or storage;
- Install host-based intrusion detection system (HIDS)/intrusion prevention system (HIPS) in web servers especially the website(s) storing or processing classified information in order to monitor suspicious activities or unauthorised creation / deletion / modification / access of files;
- Review alerts and reports from the security devices such as HIDS/HIPS to identify security attacks at the earliest possible moment. In addition, HIDS/HIPS should always be updated with latest signatures approved by change management, if appropriate;
- Do not disclose configuration information such as server software version, internal IP address, directory structure;
- Disable unnecessary modules and remove them if possible;
- Remove default or sample files from the web server software;
- Restrict web crawling for the contents that are not supposed to be searched or reached by public search engines;
- Identify important files on the web server for running the web application and protect them with proper controls such as access right control; and

- Follow the cryptographic key management requirements for servers using encryption such as TLS

Annex C: Secure Website with HTTPS Protocol

Hypertext Transfer Protocol Secure (HTTPS) is a secure communication protocol, which is one of the standardised methods for information exchange over untrusted channel and commonly adopted on website and web application. It does not only provide assurance on information confidentiality but also the authenticity of the website to address the security threats of website spoofing such as phishing website. Phishing website masquerades a trustworthy (legitimate) website and aims at acquiring classified information such as personal data. It targets to attack users who have weak security awareness by using email spoofing or instant messaging to direct them to the phishing website. Besides, unencrypted network, such as general Internet connection and insecure wireless access, allow intruders to initiate man-in-the-middle attack by capturing unencrypted data during data in motion which may compromise the data confidentiality and / or integrity.

Enablement of HTTPS for Internet facing websites can assure the authenticity and legitimacy of websites serving the citizens. Users can verify the legitimacy of the accessed websites to minimise the risk of leakage of personal data accidentally. This security measure can fight against the risk and threat of the phishing websites. The following are the considerations for adopting HTTPS protocol:

- Acquire certificate signed by reliable certification authorities (CAs) which assure the legitimacy of the website;
- Remind users to use Server Name Identification (SNI) supported browsers in order to support SNI;
- Consider using wildcard certificate to save the certificate management and maintenance effort; and
- If Content Delivery Network (CDN) service is required, B/Ds should take appropriate risk assessment and necessary protection, e.g. contractual requirement, as security measure to govern the CDN service provider.

Annex D: Web Application Secure Coding Practices

The secure coding practices are listed below:

- a) Validate all input parameters to prevent attacks such as SQL injection and cross-site scripting attacks
 - Develop a centralised module to perform the input parameter validation.
 - Check each input parameter against a strict format (i.e. whitelist) that specifies exactly which types, length, and syntax of input will be allowed.
 - Filter special characters such as “~!#\$%^&*[]<>'\r\n” from the input form, or replace them with escape sequence.
 - Do not rely exclusively on blacklist validation to detect malicious input.
 - Do not rely on client side script to perform the validation check. It should also be done at the server side.
 - Do not pass the HTML forms parameters directly to system call or database query.
 - Do not display the HTML forms parameters directly in the processing response.
- b) Sanitise application response
 - Develop a centralised module to perform the sanitisation.
 - Check all output, return codes and error codes from calls (e.g. calls to backend database) to ensure that the expected processing actually occurred.
 - Do not reveal sensitive information such as credit card number, HKID, personal telephone/mobile number, credentials and other sensitive information without proper control, e.g. masking.
 - Do not include comments about application logic in the HTML response.
 - Do not include unnecessary internal system information like internal IP address, internal host name, internal directory structure, etc. in the response.
 - Do not include verbose error messages of internal server errors (such as debug information, stack traces) to avoid exposing information to attackers. Most application/web server allows customisation of an error page in case of internal server error.
- c) HTTP trust issues
 - Do not trust and rely on HTTP REFERER headers, form fields or cookies to make security decisions as any of this data can be spoofed.
 - Do not trust these parameters from the client browser unless strong cryptographic technique is used to verify the integrity of the HTTP headers.
 - Do not pass HTTP header’s parameters directly to system call or database query.

- Do not display HTTP header's parameters directly in the processing response.
- Do not assume hidden parameters cannot be changed by users as hidden parameters can be manipulated easily by attackers.
- d) Keep sensitive session values on servers to prevent client-side modification
 - Do not put sensitive information in any client browser's cookies.
 - Use strong cryptographic techniques to protect the confidentiality and integrity of the data, if sensitive values have to be stored in client browsers.
- e) Encrypt pages with sensitive information and prevent caching
 - Encrypt pages containing sensitive information with proper algorithms and keys during transmission; e.g. TLS.
 - Use signed Java applet or ActiveX to acquire and display sensitive information.
 - Set the appropriate HTTP header attributes to prevent caching, by browser or proxy, of an individual page wherein the page contains sensitive information.
- f) Session management
 - Use a session ID that is long, complicated, and with random numbers so that it is unpredictable.
 - Set duration of session ID to as minimum as appropriate to complete the session activity.
 - Do not store session ID in URL, persistent cookies, hidden HTML field nor HTTP headers. Consider storing session ID in client browser's session cookies with proper encryption.
 - Protect session ID by TLS, so that attacker cannot sniff from the network.
 - Do not share session ID for multiple connections.
 - Do not rely on checking IP address of the incoming connection with the session ID because the IP address can be proxied.
 - Implement a logout function for the application and idle session timeout. When logging off a user or expiring the idle session, ensure that not only is the client-side cookie cleared (if possible), but also the server side session state for that browser and connections to backend servers are cleaned up.
- g) Access restriction
 - Ensure that end-user account only has the least privilege to access those functions that they are authorised, and the account has restricted access to backend database, or to run SQL or other OS commands.
 - Do not make system calls directly to real file names and directory paths. If attackers have access to source codes, they may discover system-level information. Use mapping provided by web server as a layer of filtering.
 - Do not place data file, temporary or backup files in the same directories of web servers to prevent from unauthorised access.
 - Restrict access to application and web server system or configuration files.

- Do not assume that users are unaware of special or hidden URLs or APIs.
- h) Logging
- Use POST only to send request because GET request can leave verbose information in the web/application server logs.
 - Enable web server log and transactions log such as access log about the visitors, their origins and accessed pages; agent log containing requests from web clients to the web servers; error log containing potential suspicious and abnormal activities.
 - Build a centralised module for application auditing and reporting.
- i) Use the most appropriate form of authentication methods to identify and authenticate incoming user requests.
- j) Consider using server side programming platform with strong sandbox model to protect the application server and session variables, such as Java or .Net.
- k) Protect XML data at the same way as protecting HTML traffic and do not include sensitive data in XML document in clear-text.
- l) Restrict the types of files being uploaded to the server. Uploading executable programs or scripts should be controlled.
- m) Keep abreast of the emerging risks associated with new web technologies such as Asynchronous JavaScript and XML (AJAX), JavaScript Object Notation (JSON) and HTML5.

Annex E: OWASP Secure Coding Practices Quick Reference Guide

The following information is based on the OWASP Secure Coding Practices Quick Reference Guide with customisation for fitting B/Ds' environment. In general, there are 14 areas for secure coding practices, including:

- a) Input Validation;
- b) Output Encoding;
- c) Authentication and Password Management;
- d) Session Management;
- e) Access Control;
- f) Cryptographic Practices;
- g) Error Handling and Logging;
- h) Data Protection;
- i) Communication Security;
- j) System Configuration;
- k) Database Security;
- l) File Management;
- m) Memory Management; and
- n) General Coding Practices.

The following are additional reference for B/Ds when developing web application with secure coding practices. B/Ds should consider the applicability of those practices and adopt the suitable requirements when developing the web application.

Input Validation

The first step to protect web application is validating the input information such that the information is the same as expected by design. For unexpected inputs, they should be filtered or prevented from further processing to limit potential impact. Examples of input validation include the origin of input, the character checking and data types such as number, date format, special characters or alphabetic characters. Some security considerations are highlighted as follows:

- a) Encode data to a common character set before validating;
- b) Conduct all data validation on a trusted system;
- c) Identify and classify all data sources into trusted and untrusted, and validate all data from untrusted sources (e.g., databases, file streams, etc.);
- d) Set up centralised input validation routine for the application;
- e) Specify proper character sets, such as UTF-8, for all sources of input;
- f) Reject all input that failed validation;

- g) Validate all client provided data before processing, including all parameters, URLs and HTTP header contents (e.g. cookie names and values), and including automated post backs from JavaScript, Flash or other embedded code;
- h) Validate for expected data types, data range, data length, all input against a "white" list of allowed characters whenever possible, data from redirects, and the completeness of UTF-8 decoding;
- i) Verify that header values in both requests and responses containing only ASCII characters
- j) Implement additional controls like output encoding, secure task specific APIs and account for the utilisation of the data throughout the application if any potentially hazardous characters, such as `< > " ' % ( ) & + \ \ ' \ "`, must be allowed as input;
- k) Check for null bytes (`%00`), new line characters (`%0d, %0a, \r, \n`), and "dot-dot-slash" (`../` or `..\`) path alterations characters. In cases where UTF-8 extended character set encoding is supported, address alternate representation like `%c0%ae%c0%ae/`.

Output Encoding

Output is another important area as it is a channel for disclosing information to unauthorised party. Briefly, if the output is connected to other systems, a standard and tested routing should be used and encryption should be applied where possible. The output should deliver to trusted web user or party by proper controls such as authentication and access controls. Some security considerations are highlighted as follows:

- a) Conduct all encodings on a trusted system;
- b) Utilise a standard, tested routine for each type of outbound encoding;
- c) Use contextually output encoding, if possible, for all data returned to the client that originated outside the application's trust boundary.
- d) Encode all characters unless they are known to be safe for the intended interpreter;
- e) Contextually all sanitise all output of untrusted data to queries for SQL, XML, and LDAP; and
- f) Sanitise all output of un-trusted data to operating system commands.

Authentication and Password Management

Authentication is a critical step to identify right person accessing the web application if it contains function specified for particular person. For a secure authentication system, password management is important so that a strong password policy can be enforced to protect against attacks. Some security considerations are highlighted as follows:

- a) Require authentication for all web pages and resources, except those specifically intended to be public;
- b) Enforce all authentication controls on a trusted system;
- c) Establish and utilise standard, tested, authentication services whenever possible;
- d) Use a centralised implementation for all authentication controls, including libraries that call external authentication services;
- e) Implement fail securely on all authentication controls;
- f) All administrative and account management functions must be at least as secure as planned requirement;
- g) If the application manages a credential store, ensure that only cryptographically strong one-way salted hashes of passwords are stored and that the file that stores the passwords and keys is write-able only by the application;
- h) Avoid storing password without adopting strong one-way hashing;
- i) Validate the authentication data only on completion of all data input, especially for implementations;
- j) Never indicate authentication failure responses on which part of the authentication data was incorrect;
- k) Utilise authentication for connections to external systems that involve sensitive information or functions;
- l) Encrypt and store authentication credentials for accessing services external to the application in a protected location on a trusted system;
- m) Use only HTTP POST requests to transmit authentication credentials;
- n) Only send non-temporary passwords over an encrypted connection or as encrypted data, such as in an encrypted email, except temporary passwords associated with email for reset;
- o) Enforce password complexity requirements by B/Ds.
- p) Obscure password entry on the user's screen;
- q) Enforce account disabling after an established number of invalid login attempts (e.g., five attempts is common) , and disable the account when exceeding the number of invalid login attempts ;
- r) Enforce the same level of controls for password reset and change operations as account creation and authentication;
- s) Avoid password reset questions with obvious answers (for instance, “favourite book” is a bad question because “The Bible” is a very common answer);
- t) Send to a pre-registered address with a temporary link/password if using email based reset password;
- u) Set a short expiration time to temporary passwords and links;
- v) Enforce the changing of temporary passwords on the next use;
- w) Notify users when a password is reset;
- x) Prevent password re-use;
- y) Enforce password changes based on requirements established by B/Ds such that critical systems may require more frequent changes;
- z) Disable “remember me” functionality for password fields;

- aa) Report the last use (successful or unsuccessful) of a user account to the user at their next successful login;
- bb) Implement monitoring to identify attacks against multiple user accounts with the same password. This attack pattern is used to bypass standard lockouts, when user IDs can be harvested or guessed;
- cc) Change all vendor-supplied default passwords and user IDs or disable the associated accounts;
- dd) Re-authenticate users prior to performing critical operations;
- ee) Use multi-factor authentication for highly sensitive or high value transactional accounts; and
- ff) Inspect the third party code used for authentication or conducting penetration test to verify if this authentication is not affected by known malicious codes.

Session Management

As discussed previously, session maintains authenticated user information without re-entering username/password. Therefore, if the timeout policy does not fit with business needs, attacker may re-use the session to pretend a victim to access the web application. Some security considerations are highlighted as follows:

- a) Use the server or framework's session management controls to recognise session identifiers as valid;
- b) Create random session identifier on a trusted system (e.g., the server);
- c) Set the domain and path for cookies containing authenticated session identifiers to an appropriately restricted value for the site;
- d) Terminate the associated session or connection by logout functionality;
- e) Logout functionality should be available from all authenticated pages;
- f) Establish a session inactivity timeout that is as short as possible, based on balancing risk and business functional requirements;
- g) Disallow persistent logins and enforce periodic session terminations, even when the session is active;
- h) Terminate the previous active session when establishing a new session;
- i) Generate a new session identifier on any re-authentication;
- j) Never allow concurrent logins with the same user ID unless it is one of the functional requirements in planning and design phase;
- k) Never expose session identifiers in URLs, error messages or logs;
- l) Protect server side session data from unauthorised access, by other users of the server, by implementing appropriate access controls on the server;
- m) Generate a new session identifier and deactivate the old one periodically (this can mitigate certain session hijacking scenarios where the original identifier was compromised);
- n) Generate a new session identifier if the connection security changes from HTTP to HTTPS, which can occur during authentication. Within an application, it is recommended to consistently utilise HTTPS rather than switching between HTTP to HTTPS;

- o) Utilise per-request instead of per-session strong random tokens or parameters on session management to prevent CSRF attacks;
- p) Set the “secure” attribute for cookies transmitted over an TLS connection; and
- q) Set cookies with the HttpOnly attribute, unless client-side scripts within the application to read or set a cookies value are specifically required.

Access Control

Access control is the mechanism to restrict the use of resources and/or functions in web application. For non-public resources, they are protected by access control together with proper authorisation and authentication. Some security considerations are highlighted as follows:

- a) Use only trusted system objects, such as server side session objects, for making access authorisation decisions;
- b) Use a single site-wide component to check access authorisation, including libraries that call external authorisation services;
- c) Implement fail securely for software and equipment of access controls;
- d) Deny all access if the application cannot access its security configuration information;
- e) Enforce authorisation controls on every access request, including those made by server side scripts, “includes” and requests from rich client-side technologies like AJAX and Flash;
- f) Segregate privileged logic from other application codes;
- g) Restrict access to files or other resources, such as protected URLs, functions, direct object references, services, and security-relevant configuration information, to authorised users only;
- h) Use encryption and integrity checking on the server side to catch state tampering if data must be stored on the client side;
- i) Enforce application logic flows to comply with business rules;
- j) Limit the number of transactions, on a single user or device usage in a given period in order to increase the difficulty in performing DoS attack;
- k) Never rely on checking the “referrer” header only as the sole authorisation check because it can be spoofed;
- l) Periodically re-validate an user’s authorisation for long authenticated sessions to prevent unauthorised person using the non-logout session;
- m) Implement account auditing and enforce the disabling of unused accounts after the expiration period;
- n) Disable accounts and terminate sessions when authorisation ceases due to changes such as changes to role, employment status, business process;
- o) Set up service accounts or accounts supporting connections to or from external systems by the least privilege possible principle;
- p) Create access control policy and procedures according to application’s business rules, data types and access authorisation criteria and/or processes so that access can be properly provisioned and controlled; and

- q) Most database management systems allow users to be classified such that individual users may be enabled to merely access data, or to perform a certain limited function. This permission can be given with respect to a whole database, or even to selected fields of a database. Granularity of access should be added to database access control by the use of logical “views” so that the user views only the part of the database he/she is authorised to access. User profiles should be well protected and should not be accessed by unauthorised persons.

Cryptographic Practices

It is necessary to protect classified information using encryption. Cryptographic practices provide guidance for web application developers such as key length, encryption/decryption algorithms and key management. Some security considerations are highlighted as follows:

- a) Follow the encryption requirements mentioned in Section 12.1 of IT Security Guidelines to protect the classified data. That is, for information classified as CONFIDENTIAL or above, the symmetric encryption key length shall be at least 128-bit for the AES encryption or the equivalent, whereas the asymmetric encryption key length shall be at least 2048-bit for the RSA encryption. Alternatively, the requirement can be met by Elliptic Curve Cryptography (ECC) encryption with key length of 224-bit;
- b) Use random number generator to generate encryption keys, random numbers, random file names, and random strings, in order to avoid guessing by malicious persons; and
- c) Implement fail securely for cryptographic modules/equipment.

Error Handling and Logging

There is no application going perfect without problem or error, error handling and logging provide useful information for developer on web application troubleshooting. However, if attacker obtains such information, it may become a fruitful source for intrusion. Therefore, the information should be sufficient for troubleshooting but not excessive. Some security considerations are highlighted as follows:

- a) Avoid displaying debugging or stack of trace information when using error handlers;
- b) Release allocated memory when error conditions occur;
- c) Deny access by default should be adopted for error handling logic associated with security controls;
- d) Ensure logs contain important log event data;
- e) Ensure log entries that include untrusted data will not be executed as code in the intended log viewing interface or software;
- f) Restrict access to logs to only authorised personnel;

- g) Utilise a master routine for all logging operations;
- h) Prevent to store sensitive information in logs, including unnecessary system details, session identifiers or passwords;
- i) Ensure that logs are reviewed, analysed and escalated if any issue is identified;
- j) Log all input validation failures, authentication attempts, access control failures, apparent tampering events, attempts to connect with invalid or expired session tokens, system exceptions, administrative functions, backend TLS connection failures, and cryptographic module failures; and
- k) Use a cryptographic hash function to validate log entry integrity.

Data Protection

It is important that B/Ds shall follow the requirements on identifying and protecting government classified data during storing, processing and transmission. Some security considerations are highlighted as follows:

- a) Implement least privilege, restricting users to only the functionality, data and system information on need-to-use basis;
- b) Protect all cached or temporary copies of classified data stored on the server from unauthorised access and purge those temporary working files as soon as they are no longer required;
- c) Apply data encryption according to its classification;
- d) Protect server-side source-code from being downloaded by a user;
- e) Never store passwords, connection strings or other sensitive information in clear text or in any non-cryptographically secure manner on the client side;
- f) Remove comments in user accessible production code that may reveal backend system or other sensitive information;
- g) Avoid transmitting sensitive information using HTTP GET parameters;
- h) Disable auto-complete features on forms expected to contain sensitive information including authentication;
- i) Disable client side caching on pages containing sensitive information;
- j) Remove classified data when it is no longer required;
- k) Use fake data during development phase and if production data is required due to sufficient supporting, management approval is required; and
- l) Implement appropriate access controls for sensitive data stored on the server, including cached data, temporary files.

Communication Security

Communication is one of the core components of web application, as generally all websites / web applications are connected through networks, regardless whether they are trusted or untrusted network. Proper security controls are required if data are transmitted over untrusted network and the web application contains government classified information. Some security considerations are highlighted as follows:

- a) Implement encryption for the transmission of information according to its classification;
- b) Validate the correct domain name by TLS certificates;
- c) Never fall back an insecure connection when TLS connection is failed;
- d) Utilise TLS connections for all contents requiring authenticated access and containing classified information;
- e) Utilise TLS for connections to external systems that involve classified information or functions;
- f) Utilise a single standard TLS implementation that is configured appropriately;
- g) Specify character encodings for all connections; and
- h) Filter parameters containing sensitive information from the HTTP reference when linking to external sites.

System Configuration

If the web hosting system is not configured in a secure manner, attacker may successfully compromise the system and bypass security measures in web application. It is necessary to harden system and define system configuration baseline for achieving security requirement. Some security considerations are highlighted as follows:

- a) Apply latest and tested patches on IT system;
- b) Disable directory listings;
- c) Restrict the web server, process and service accounts to the least privileges if possible;
- d) Fail securely when exceptions occur;
- e) Remove all unnecessary functionality and files;
- f) Remove test code not intended for production prior to deployment;
- g) Prevent disclosure of the directory structure in the robots.txt file by placing directories not intended for public indexing into an isolated parent directory, and disable the user agent's access of the entire parent directory;
- h) Define which HTTP method, Get or Post, the application will support and whether it will be handled differently in different pages in the application;
- i) Disable unnecessary HTTP methods, such as WebDAV extensions.
- j) Utilise a well-vetted authentication mechanism if an extended HTTP method that supports file handling is required;
- k) Remove unnecessary information from HTTP response headers related to the OS, web-server version and application frameworks;
- l) The security configuration store for the application should be able to be output in human readable format to support auditing;
- m) Isolate development environments from the production network and provide access only to authorised development and test groups; and

- n) Follow change control process when making changes on both development and production.

Database Security

Similar to data protection, database is widely used in web application development. In addition, it may act as one of the authentication sources. Therefore, if database is being compromised, the data and user credentials (e.g. username and password) can be retrieved by attackers. Some security considerations are highlighted as follows:

- a) Use strongly typed parameterised queries in order to prevent injection attack;
- b) Utilise input validation and output encoding to scan the input meta characters before executing the database command;
- c) Ensure that variables are strongly typed (or generating error, refusing to compile if the argument passed to a function does not closely match with the expected type);
- d) Use the lowest possible level of privilege and secure credentials when application is accessing the database;
- e) Never hard-code the connection strings within the application, but storing these in a separate encrypted configuration file on a trusted system instead;
- f) Use stored procedures to abstract data access and allow for the removal of permissions to the base tables in the database;
- g) Close the connection as soon as possible;
- h) Remove or change all default database administrative passwords with strong passwords/phrases or implement multi-factor authentication instead;
- i) Turn off all unnecessary database functionality;
- j) Remove unnecessary default vendor contents (e.g., sample schemas);
- k) Disable any default accounts that are not required to support business requirements; and
- l) Use unique credentials for every trusted distinction (e.g., user, read-only user, guest, administrators) connecting to the database.

File Management

A web application may involves many files; if there is a lack of security control for relevant files such as improper access control, file execution and update control, attacker can update web application with malicious code such as Trojan or malware. Some security considerations are highlighted as follows:

- a) Never pass user supplied data directly to any dynamic included function;
- b) Require authentication before allowing a file to be uploaded;
- c) Limit the types of file that can be uploaded to those required by business use;
- d) Ensure application files and resources are read-only;

- e) Scan user uploaded files for malware;
- f) Validate uploaded file types by checking file headers instead of by extension alone;
- g) Prevent or restrict the uploading of any file that may be interpreted by the web server;
- h) Turn off execution privileges on file upload directories;
- i) Implement safe uploading in UNIX by mounting the targeted file directory as a logical drive using the associated path or the “chrooted” environment;
- j) Use a white list of allowed file names and types when referencing existing files; validate and reject files that do not match with the expected values;
- k) Never pass user supplied data into a dynamic redirect, or perform such redirection with validated relative path URLs only;
- l) Never pass directory or file paths, but use index values mapped to pre-defined list of paths instead; and
- m) Never send the absolute file path to the client.

Memory Management

Similar to file management, if there is a lack of security control for memory running web application such as input and output control, shared resources, or poor garbage collection, attacker may execute malicious process for website and retrieve information from system memory running the web application. Some security considerations are highlighted as follows:

- a) Utilise input and output control for untrusted data;
- b) When calling functions that accept a number of bytes to copy such as `strncpy()`, beware that if the destination buffer size is equal to the source buffer size, it may not NULL-terminate the string;
- c) Check buffer boundaries if calling the function in a loop and make sure there is no danger of writing past the allocated space;
- d) Truncate all input strings to a reasonable length before passing them to the copy and concatenation functions;
- e) Make sure to close resources specifically and do not rely on garbage collection (e.g., connection objects, file handles, etc.);
- f) Use non-executable stacks when available in order to prevent the stack and heap memory areas from being executed, and hence prevent certain buffer overflow exploits;
- g) Understand and avoid the use of known vulnerable functions from the programming languages (e.g., `printf`, `strcat`, `strcpy` etc.) ; and
- h) Release allocated memory properly upon completion of functions and at all exit points.

General Coding Practices

In addition to the security considerations mentioned above, some general practices to address the IT security concerns are shown below:

- a) Use tested managed code rather than unmanaged code to avoid typical programming mistakes that may lead to security loopholes and unstable applications;
- b) Utilise task specific built-in APIs to conduct operating system tasks and prevent the application to issue commands directly to the operating system, especially through the use of application initiated command shells;
- c) Use checksums or hashes to verify the integrity of program source codes, libraries, executables and configuration files;
- d) Utilise locking to prevent multiple simultaneous requests or use a synchronisation mechanism to prevent race conditions;
- e) Protect shared variables and resources from inappropriate concurrent access;
- f) Explicitly initialise all variables and other data stores, either during declaration or just before the first usage;
- g) Raise privileges as late as possible and drop them as soon as possible in cases that application must run with elevated privileges;
- h) Minimise security issues (for example, buffer overflow attack) due to programming bugs by,
 - Avoiding calculation errors by understanding the programming syntax and how it interacts with numeric calculation; and
 - Paying close attention to byte size discrepancies, precision, signed/unsigned distinctions, truncation, conversion and casting between types, non-numeric calculations, and how the syntax handles numbers that are too large or too small for its underlying representation;
- i) Restrict users from generating new code or altering existing code without going through change management process;
- j) Review and test all secondary applications, third party code and libraries in order to avoid new vulnerabilities; and
- k) Implement safe updating. Use cryptographic signatures for code with verification of those signatures by the download clients if the application will utilise automatic updates. Download the code through the encrypted channels.

The 14 secure coding practices described above should be considered during the development phase. A comprehensive checklist can be obtained from the official website of OWASP.

Link:

https://www.owasp.org/index.php/OWASP_Secure_Coding_Practices_-_Quick_Reference_Guide

